

Výstupy ŠVP

INF-07a Ve vztahu k charakteru a velikosti vstupu hodnotím nároky algoritmů; porovnám algoritmy podle různých hledisek a vyberu pro řešený problém ten nejvhodnější. (*Informatika 4, 2/2 · z INF-07*)

INF-07b Vylepším algoritmus podle zvoleného hlediska a zobecním řešení pro širší třídu problémů. (*Informatika 4, 2/2 · z INF-07*)

INF-08d Ve snaze o vyšší efektivitu navrhuji, řídím a hodnotím souběh procesů. (*Informatika 4, 2/2 · z INF-08*)

Učivo: testování, optimalizace – vliv vstupních dat na spotřebované výpočetní zdroje

Lehké úlohy procvičí vyhledávání, řazení a pojem souběhu na malých příkladech; ve středních porovnáváš a vylepšuješ algoritmy; těžké počítají porovnání a ukazují, proč na volbě algoritmu záleží. Pojmy a vzorce máš v listu Must know. Vyber si, kolik úloh spočítáš; řešení si ukládej do svého portfolia.

Lehké ★☆☆

1. Lineární vyhledávání postupně porovnává prvky seznamu [7, 2, 9, 4, 6] s hledaným číslem. Kolik porovnání provede při hledání 4 a kolik při hledání 5? Jaký je nejlepší a nejhorší případ pro seznam s n prvky?
2. Na který seznam lze použít binární vyhledávání (půlení)? a) [3, 7, 7, 12, 30]; b) ["Adam", "Eva", "Bára"]; c) seznam příjmení seřazený podle abecedy; d) [9, 5, 2, 1]. Zdůvodni.
3. Řazení vkládáním bere prvky zleva a každý zařadí na správné místo mezi už seřazené prvky před ním. Zapiš seznam [6, 2, 5, 1] po každém zařazení.
4. Algoritmus s počtem kroků úměrným n^2 zpracuje 1 000 prvků za 1 s. Jak dlouho mu potrvá 2 000 prvků a jak dlouho 10 000 prvků?
5. Petr drží pravítko a čeká, až se uvolní kružítko; Jana drží kružítko a čeká na pravítko. Co se stane? Jak se tento stav nazývá? Navrhni pravidlo, které mu zabrání.
6. Řazení výběrem porovná při hledání minima každý prvek nesetříděné části, celkem $\frac{n(n-1)}{2}$ porovnání. Kolik porovnání potřebuje pro 5, 10 a 100 prvků? Záleží na tom, jak byl seznam na začátku uspořádán?

Střední ★★☆☆

7. Slévání: dva seřazené seznamy [2, 5, 9] a [1, 6, 7, 10] spoj do jednoho seřazeného tak, že vždy porovnáš první nepoužité prvky obou seznamů a menší přesuneš do výsledku. Kolik porovnání provedeš? Kolik nejvyšší porovnání je potřeba pro seznamy délek a a b ?
8. Znamky [3, 1, 2, 1, 5, 3, 1] seřaď bez porovnávání: spočítej četnost každé známky 1 až 5 a pak známky vypiš podle četností. Kolik průchodů seznamem je potřeba? Proč tento postup nejde použít na libovolná čísla?
9. Uprav binární vyhledávání tak, aby vrátilo index prvního výskytu hledaného čísla (v seznamu se čísla mohou opakovat). Vyzkoušej hledání 7 a 8 v seznamu [1, 3, 7, 7, 7, 9].
10. Program změřil dobu řazení: 1 000 prvků 0,05 s; 2 000 prvků 0,2 s; 4 000 prvků 0,8 s. Jak roste doba s velikostí vstupu? Jak dlouho potrvá 64 000 prvků?
11. Máš sečíst 1 024 čísel. Jeden procesor potřebuje 1 023 sčítání za sebou. Když máš procesorů libovolně mnoho, v každém kole sečtou čísla po dvojicích současně. Kolik kol je potřeba? Kolik procesorů pracuje v prvním kole?
12. Bublínkové řazení s vylepšením (skončí po průchodu bez výměny): kolik průchodů potřebuje seznam [2, 3, 4, 5, 1] a kolik [5, 1, 2, 3, 4]? Proč se tak liší, i když mají stejná čísla?

Těžké ★★★

13. Řazení vkládáním: kolik porovnání provede pro už seřazený seznam 1 000 čísel a kolik pro seznam seřazený obráceně? Porovnej s řazením výběrem. Kdy je řazení vkládáním dobrá volba?

14. Co program vypíše? Proč je největší počet kroků právě 10? Kolik kroků potřebuje neúspěšné hledání a proč?

```
def binarni(s, x):
    lo, hi, kroky = 0, len(s) - 1, 0
    while lo <= hi:
        m = (lo + hi) // 2
        kroky = kroky + 1
        if s[m] == x:
            return m, kroky
        elif s[m] < x:
            lo = m + 1
        else:
            hi = m - 1
    return -1, kroky

s = list(range(1, 1024))
print(len(s), max(binarni(s, x)[1] for x in s))
print(binarni(s, 512))
print(binarni(s, 0))
```

15. Sedm úloh s délkami 8, 7, 6, 5, 4, 3, 3 min rozděl na 3 procesory. Hladový postup: úlohy seřaď od nejdelší a každou dej procesoru, který je zrovna nejméně vytížený. Za jak dlouho skončí? Existuje lepší rozdělení?
16. V seřazeném seznamu [1, 4, 6, 9, 11, 15] hledej dvojici se součtem 20. Hrubá síla zkouší všechny dvojice. Vylepšení: ukazovátko vlevo a vpravo; je-li součet menší, posuň levé doprava, je-li větší, posuň pravé doleva. Kolik součtů spočítá každý postup? Jak rostou pro n prvků?
17. Hledání maxima i minima zvlášť potřebuje $2(n - 1)$ porovnání. Vylepšení: ber prvky po dvojicích, porovnej je mezi sebou a menší porovnej jen s minimem, větší jen s maximem. Kolik porovnání potřebuje každý postup pro 100 prvků?
18. Propojení s H34: Řazení porovnáváním musí rozlišit všech $n!$ možných pořadí vstupu a každé porovnání dá odpověď ano/ne. Kolik porovnání nejméně potřebuje v nejhorším případě pro 5 a pro 10 prvků? Porovnej s řazením výběrem.

Výsledky ke kontrole

1. $4 \rightarrow 4$ porovnání; $5 \rightarrow 5$ porovnání (projde celý seznam a nenajde); nejlépe 1 porovnání (hledaný prvek je první), nejhorší n
2. a) ano (seřazený, opakování nevadí); b) ne – není seřazený; c) ano (texty se porovnávají abecedně); d) ano, ale seřazený sestupně – porovnání se musí obrátit
3. $[2, 6, 5, 1] \rightarrow [2, 5, 6, 1] \rightarrow [1, 2, 5, 6]$ (tři kroky; seřazená část roste zleva)
4. dvojnásobný vstup $\rightarrow 2^2 = 4 \times$ víc, tj. 4 s; desetinásobný $\rightarrow 100 \times$, tj. 100 s
5. oba budou čekat donekonečna – uváznutí (deadlock); pravidlo: pomůcky se berou vždy ve stejném pořadí (nejdřív pravítko, pak kružítko), nebo kdo nedostane druhou pomůcku, vrátí i první
6. 10; 45; 4 950; nezáleží – řazení výběrem porovnává vždy stejně, i když je seznam už seřazený
7. výsledek $[1, 2, 5, 6, 7, 9, 10]$, 6 porovnání (zbytek $[10]$ se připojí bez porovnání); nejvýš $a + b - 1$ – slévání je rychlé, proto na něm stojí řazení sléváním
8. četnosti 1: 3, 2: 1, 3: 2, 4: 0, 5: 1 $\rightarrow [1, 1, 1, 2, 3, 3, 5]$; stačí jeden průchod seznamem a jeden seznamem četností; funguje jen pro malý rozsah celých hodnot (pro čísla do miliardy by seznam četností byl obrovský)
9. když $s[m] == x$, ulož m jako kandidáta a hledej dál vlevo ($hi = m - 1$); $7 \rightarrow$ index 2; $8 \rightarrow -1$ (není v seznamu); počet kroků zůstane asi $\log_2 n$
10. při zdvojnásobení n roste čas $4 \times \rightarrow$ kvadraticky (n^2); 64 000 je $64 \times$ víc $\rightarrow 64^2 = 4096 \times$ déle, asi 204,8 s $\approx$ 3,4 min
11. 10 kol ($1\ 024 \rightarrow 512 \rightarrow 256 \rightarrow \dots \rightarrow 1$, $2^{10} = 1024$); v prvním kole 512 procesorů; sčítání je pořád 1 023, ale díky souběhu trvají jen 10 kroků
12. první 5 průchodů (4 s výměnou + kontrolní), druhý jen 2; velký prvek „probublá“ doprava v jednom průchodu, ale malý prvek se posune doleva jen o jedno místo za průchod
13. seřazený: 999 porovnání (každý prvek jen jednou porovná se sousedem); obrácený: 499 500 (jako výběr, $\frac{n(n-1)}{2}$); výběr potřebuje vždy 499 500; vkládání je výhodné pro téměř seřazená data
14. $1023\ 10 / (511, 1) / (-1, 10)$; $1023 = 2^{10} - 1$ prvků, každý krok interval zhruba způlí $\rightarrow$ nejvýš 10 kroků; 512 je přesně uprostřed (1 krok); neúspěšné hledání (0) potřebuje plných 10 kroků – interval se musí zmenšovat, dokud není prázdný
15. hladově: $P1\ 8 + 3 + 3 = 14$, $P2\ 7 + 4 = 11$, $P3\ 6 + 5 = 11 \rightarrow 14$ min; lepší rozdělení $\{8, 4\}$, $\{7, 5\}$, $\{6, 3, 3\} \rightarrow 12$ min, víc zkrátit nelze ($\frac{36}{3} = 12$); hladový postup je rychlý, ale ne vždy optimální
16. dvojice (9, 11); hrubá síla nejvýš $\binom{6}{2} = 15$ součtů, ukazovátka jen 5 součtů ($1 + 15$, $4 + 15$, $6 + 15$, $6 + 11$, $9 + 11$); hrubá síla roste jako n^2 , ukazovátka jen jako n
17. zvlášť 198; po dvojicích 148 (první dvojice 1 porovnání, každá další 3: $1 + 49 \cdot 3 = 148$), tj. asi $\frac{3n}{2}$ – úspora asi čtvrtiny
18. nejméně $\log_2 n!$: pro 5 prvků $\log_2 120 \doteq 6,9 \rightarrow 7$ porovnání (výběr 10); pro 10 prvků $\log_2 3\ 628\ 800 \doteq 21,8 \rightarrow 22$ porovnání (výběr 45); nejrychlejší algoritmy potřebují asi $n \log_2 n$ porovnání